

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CEF-2113 – Algorithms and Data Structures Lab
Laboratory Manual

Department	:	School of Basic Science, Humanities and Management
Course Name & code	:	CEF 2113 & Algorithms and Data Structures Lab
Semester & branch	:	III Sem & BTech Computer Science & Finance Technology
Name of the faculty	:	Dr. Manjunatha

No of contact hours/week:

L	T	P	C
0	0	3	1

CONTENTS

Lab No.	Title	Page No
	Course Objectives	
	Evaluation Plan	
	Instructions to Students	
1	Mapping of 2-D arrays to 1-D arrays	
2	Solving problems using recursion	
3	Stacks and its applications	
4	Queues and its applications	
5	Singly linked list and its applications	
6	Doubly linked list and circular linked list operations	
7	Trees and their applications	
8	Graph representations and traversal	
9	Algorithm design Technique: Brute force method	
10	Algorithm design Technique: Divide and conquer method	
11	Algorithm design Technique: Dynamic Programming	
12	Algorithm design Technique: Greedy Techniques	

Course Objectives ·

1. Design, develop, and implement programs using the concepts of Recursion and the mapping of 2D arrays to 1D arrays
2. Design, develop, and implement programs and applications using linear data structures such as Stacks, Queues, and Linked Lists
3. Design, Develop, and Implement programs and applications using the non-Linear Data structures such as trees and graphs
4. Application of different algorithm design techniques .

Evaluation plan : (Tentative)

- Internal Assessment Marks: 60%
- End-semester assessment of 2-hour duration: 40 %

Evaluation pattern

Internal Marks – 60 + End Sem - 40	
Internal Marks	Internal Assessment – 40 + Mid-sem - 20
Internal Assessment	Lab observations – 2*12(24) + Viva (16)
Lab Observation	Record (1*12) + Execution (1*12)
Viva	Quiz(16)

INSTRUCTIONS TO THE STUDENTS

Pre-Lab Session Instructions

- Be on time, adhere to the institution's rules, and maintain decorum.
- Leave your mobile phones, pen drives, and other electronic devices in your bag and keep the bag in the designated place in the lab.
- Must Sign in to the log register provided.
- Make sure to occupy the allotted system and answer the attendance.

In-Lab Session Instructions

- Follow the instructions on the allotted exercises.
- Show the program and results to the instructors on completion of experiments.
- Copy the program and results for the lab record.
- Prescribed textbooks and class notes can be kept ready for reference if required.

General Instructions for the Exercises in Lab

- Academic honesty is required in all your work. You must solve all programming assignments independently, except where group work is authorized. This means you must not take, show, give, or otherwise allow others to take your program code, problem solutions, or other work.
- The programs should meet the following criteria:
    - Programs should be interactive with appropriate prompt messages, error messages if any, and descriptive messages for outputs.
    - Programs should perform input validation (Data type, range error, etc.), give appropriate

error messages, and suggest corrective actions.

- Comments should be used to give the statement of the problem, and every function should indicate the purpose of the function, inputs, and outputs.
- Statements within the program should be properly indented.
- Use meaningful names for variables and functions.
- Make use of constants and type definitions wherever needed.
- The exercises for each week are divided into three sets:
  - Solved solutions
  - Lab exercises - to be completed during lab hours
  - Additional Exercises - to be completed outside the lab or in the lab to enhance the skill

Questions for lab tests and examinations are not necessarily limited to the questions in the manual but may involve some variations and/or combinations of the questions.

THE STUDENTS SHOULD NOT

- Possess mobile phones or any other electronic gadgets during lab hours.
- Go out of the lab without permission.
- **Change/update any configuration in your allotted system. If so, the student will lose internal assessment marks**

Week 1 - Mapping of 2-D Arrays to 1-D Arrays in Java

Learning Objectives

After completing this experiment, students will be able to:

1. Understand the memory representation of 2-D arrays.
2. Convert a 2-D array into a 1-D array using Row-Major Mapping.
3. Convert a 2-D array into a 1-D array using Column-Major Mapping.
4. Access elements using mapping formulas.
5. Apply array mapping concepts in data structure applications.

Two-Dimensional Array

A two-dimensional array consists of rows and columns.

Example:

```
1  2  3
4  5  6
7  8  9
```

Declaration:

```
int[][] arr =
{
        {1,2,3},
        {4,5,6},
        {7,8,9}
};
```

Why Mapping is Required?

Computer memory is linear in nature. Therefore, multidimensional arrays are stored internally as one-dimensional sequences.

Mapping converts a 2-D array into a 1-D array representation.

Example:

2-D Array

```
1 2 3
4 5 6
7 8 9
```

1-D Representation

1 2 3 4 5 6 7 8 9

Row-Major Mapping

In Row-Major Mapping, elements are stored row by row.

Formula

For an element at position:

A[i][j]

The corresponding index in the 1-D array is:

Index = i × Number_of_Columns + j

Example

For

1 2 3

4 5 6

7 8 9

Mapping becomes:

1 2 3 4 5 6 7 8 9

Column-Major Mapping

Formula

Index = j × Number_of_Rows + i

Example

For

1 2 3

4 5 6

7 8 9

Mapping becomes:

1 4 7 2 5 8 3 6 9

Algorithm (Row-Major Mapping)

1. Read the number of rows and columns.
2. Create a 2-D array.
3. Accept elements from the user.
4. Create a 1-D array of size rows × columns.
5. Traverse the 2-D array row-wise.
6. Store elements into the 1-D array.
7. Display the mapped array.

Sample Program

Program: Convert 2-D Array into 1-D Array Using Row-Major Mapping

```
import java.util.Scanner;
public class RowMajorMapping
{
        public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of rows: ");
int rows = sc.nextInt();
System.out.print("Enter number of columns: ");
int cols = sc.nextInt();
int[][] arr = new int[rows][cols];
int[] oneD = new int[rows * cols];
System.out.println("Enter array elements:");
for(int i = 0; i < rows; i++)
{
        for(int j = 0; j < cols; j++)
        {
                arr[i][j] = sc.nextInt();
        }
}
```

```
int k = 0;
for(int i = 0; i < rows; i++)
{
        for(int j = 0; j < cols; j++)
        {
                oneD[k++] = arr[i][j];
        }
}
System.out.println("Mapped 1-D Array:");
for(int value : oneD)
{
        System.out.print(value + " ");
}
}
}
```

Sample Output

Enter number of rows: 3

Enter number of columns: 3

Enter array elements:

1 2 3

4 5 6

7 8 9

Mapped 1-D Array:

1 2 3 4 5 6 7 8 9

Week 1: Exercises:

1. Write a Java program to convert a 2-D array into a 1-D array using Column-Major Mapping.

   ### Expected Output

   Original Array
   1 2 3
   4 5 6
   7 8 9

   Column Major Mapping
   1 4 7 2 5 8 3 6 9

2. Write a Java program to calculate the 1-D index position of an element present at row i and column j using the Row-Major Mapping formula.

   ### Example

   Rows = 4
   Columns = 5

   Element Position = (2,3)

   Index = 2 × 5 + 3
   Index = 13

   ### Expected Output

   Mapped Index = 13

3. Write a Java program to convert a given 1-D array back into a 2-D array of specified dimensions.

   ### Example Input

   1-D Array:
   10 20 30 40 50 60

   Rows = 2
   Columns = 3

   ### Expected Output

   10 20 30
   40 50 60

4. Store two matrices in 1-D arrays using Row-Major Mapping and perform matrix addition.

   ### Example

   Matrix A
   1 2

3 4
Matrix B
5 6
7 8
Expected Output
Sum Matrix

6 8
10 12

5. Write a Java program to read a matrix and determine whether it is a sparse matrix (contains more zeros than non-zero elements). Store all non-zero elements in a 1-D array using the format: row column value

Example Input

0 0 3
0 5 0
7 0 0

Expected Output

Sparse Matrix Representation
0 2 3
1 1 5
2 0 7

6. Write a Java program to find an element in a 2-D array and display its corresponding 1-D mapped index using Row-Major Mapping.

Example Input

Matrix
10 20 30
40 50 60
70 80 90
Search Element: 60

Expected Output

Element Found at:
Row = 1
Column = 2
Mapped Index = 5

Formula Used

Index = (Row × Number_of_Columns) + Column

Additional Exercises:

7. Write a Java program to:
    - Read a matrix.
    - Store it in a 1-D array using Row-Major Mapping.
    - Generate the transpose of the matrix.
    - Display the transposed matrix.
8. Write a Java program to:
    - Read two matrices.
    - Store both matrices as 1-D arrays using Row-Major Mapping.
    - Perform matrix multiplication.
    - Display the resultant matrix.

Week-2: Solving Problems Using Recursion in Java

Aim: To understand the concept of recursion and develop Java programs to solve computational problems using recursive techniques.

Learning Objectives

After completing this laboratory, students will be able to:

- Understand recursive problem-solving techniques.
- Differentiate between recursion and iteration.
- Implement recursive functions in Java.
- Analyze recursive calls and execution flow.
- Solve mathematical and data structure problems using recursion.

Prerequisites

Students should be familiar with:

- Java methods
- Conditional statements
- Loops
- Function calls
- Basic mathematical computations

What is Recursion?

Recursion is a programming technique in which a method calls itself repeatedly until a terminating condition is reached.

A recursive program consists of:

1. Base Case

Stops the recursion.

2. Recursive Case

Calls itself with a smaller or simpler problem.

General Structure

```
returnType function(parameters)
{
if(base_condition)
return value;

return function(smaller_problem);
}
```

Why Recursion?

Recursion is useful for:

- Tree traversal

- Graph traversal
- Divide and Conquer algorithms
- Backtracking problems
- Mathematical computations

Examples:

- Factorial
- Fibonacci Series
- Tower of Hanoi
- Binary Search
- Tree Traversal

Example: Recursive Factorial Algorithm

1. Read a number n.
2. If n is 0 or 1, return 1.
3. Otherwise return n × factorial(n−1).
4. Display the result.

Program: Factorial Using Recursion

```
import java.util.Scanner;
public class RecursiveFactorial
{
        static long factorial(int n)
{
if(n == 0 || n == 1)
        return 1;
return n * factorial(n - 1);
}

public static void main(String[] args)
{
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println("Factorial = " + factorial(n));
}
}
```

Sample Output

```
Enter a number: 5
2
Factorial = 120
```

Week 2: Exercises:

1. Write a Java program to generate Fibonacci numbers using recursion.
2. Write a Java program to find the sum of digits of a given number using recursion.
3. Write a Java program to reverse a given integer using recursion.
4. Write a Java program to find the GCD using Euclid's Recursive Algorithm.

Additional Exercises:

1. Write a Java program to search for an element in a sorted array using recursive binary search.
2. Write a Java program to compute x^n using recursion.

Week-3: Stacks and its Applications

Learning Objectives

At the end of this laboratory session, students will be able to:

1. Understand the Stack Abstract Data Type (ADT).
2. Implement stack operations using arrays and Java collections.
3. Apply stack concepts to solve computational problems.
4. Develop programs using stack-based algorithms.
5. Analyze the time complexity of stack operations.
6. Use stacks in expression evaluation and string processing applications.

Stack

A **Stack** is a linear data structure that follows the **LIFO (Last In First Out)** principle.

Basic Operations

- **push()** : Insert an element at the top.
- **pop()** : Remove the top element.
- **peek()** : View top element.
- **isEmpty()** : Check whether stack is empty.
- **size()** : Number of elements.

Applications of Stack

- Function call management
- Expression conversion
- Expression evaluation
- Parentheses matching
- Undo operations
- Browser navigation
- Backtracking algorithms
- Depth First Search (DFS)

Sample Exercise

Program : Implementation of Stack Using Array

Algorithm

1. Create a stack using an array.
2. Initialize top = -1.
3. Display a menu with Stack operations.
4. Accept user's choice.
5. Perform Push, Pop, Peek, or Display operation.
6. Repeat until the user chooses Exit.

Program:

```
import java.util.Scanner;

class StackArray {
    int stack[];
    int top;

    StackArray(int size) {
        stack = new int[size];
        top = -1;
    }

    void push(int value) {
        if (top == stack.length - 1) {
            System.out.println("Stack Overflow");
        } else {
            stack[++top] = value;
            System.out.println(value + " inserted into stack.");
        }
    }

    void pop() {
        if (top == -1) {
            System.out.println("Stack Underflow");
        } else {
            System.out.println("Deleted Element: " + stack[top--]);
        }
    }

    void peek() {
        if (top == -1) {
            System.out.println("Stack is Empty");
        } else {
            System.out.println("Top Element: " + stack[top]);
        }
    }

    void display() {
        if (top == -1) {
            System.out.println("Stack is Empty");
        } else {
            System.out.println("\nStack Elements:");
            for (int i = top; i >= 0; i--) {
```

```
            System.out.println(stack[i]);
        }
    }
  }
}

public class StackDemo {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter Stack Size: ");
      int size = sc.nextInt();

      StackArray s = new StackArray(size);

      int choice, value;

      do {
         System.out.println("\n----- STACK MENU -----");
         System.out.println("1. Push");
         System.out.println("2. Pop");
         System.out.println("3. Peek");
         System.out.println("4. Display");
         System.out.println("5. Exit");
         System.out.print("Enter your choice: ");

         choice = sc.nextInt();

         switch (choice) {

            case 1:
               System.out.print("Enter value to push: ");
               value = sc.nextInt();
               s.push(value);
               break;

            case 2:
               s.pop();
               break;

            case 3:
               s.peek();
```

```
                    break;

                case 4:
                    s.display();
                    break;

                case 5:
                    System.out.println("Exiting Program...");
                    break;

                default:
                    System.out.println("Invalid Choice!");
            }

        } while (choice != 5);

        sc.close();
    }
}
```

Output:

```
Enter Stack Size: 5

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1

Enter value to push: 10
10 inserted into stack.

Enter your choice: 1
Enter value to push: 20
20 inserted into stack.

Enter your choice: 1
Enter value to push: 30
30 inserted into stack.

Enter your choice: 4
```

Stack Elements:
30
20
10

Enter your choice: 3
Top Element: 30

Enter your choice: 2
Deleted Element: 30

Enter your choice: 4

Stack Elements:
20
10

Enter your choice: 5
Exiting Program...

Week-3 Exercises:

1. Write a Java program that accepts a string from the user and reverses it using a stack.

2. Write a Java program that accepts an arithmetic expression containing the symbols (), {}, and [], and determines whether the expression is balanced using a stack.

3. Write a Java program that accepts an infix expression and converts it into an equivalent postfix expression using stack operations while considering operator precedence and associativity.

4. Write a Java program that accepts a postfix expression consisting of single-digit operands and arithmetic operators (+, -, *, /) and evaluates the expression using a stack.

5. Write a Java program that accepts a decimal number from the user and converts it into its binary representation using a stack.

Additional Exercises:

1. Write a Java program to implement two stacks using a single array. The program should provide separate operations for both stacks, including:
   Push an element into Stack 1
   Push an element into Stack 2

Pop an element from Stack 1
Pop an element from Stack 2
Display the contents of Stack 1
Display the contents of Stack 2
Check for overflow and underflow conditions

2. Given an array of integers, the Next Greater Element for an element is the first element on its right side that is greater than it. If no such element exists, the NGE is considered **-1**. Write a Java program to find the next greater element for each element in the array using a stack-based approach.

   Input: [4, 5, 2, 25, 7, 8]
   Output:

Element	Next Greater Element
4	5
5	25
2	25
25	-1
7	8
8	-1

Week-4: Queue and its Applications

Learning Outcomes

At the end of this laboratory session, students will be able to:

1. Understand the FIFO (First In First Out) principle of queues.
2. Implement queue operations using Java collections.
3. Apply queues to solve practical problems.
4. Analyze the time complexity of queue operations.
5. Develop problem-solving skills using queue-based approaches.

A **Queue** is a linear data structure that follows the **First In First Out (FIFO)** principle. The element inserted first will be removed first.

Common Queue Operations

- **Enqueue**: Insert an element at the rear.
- **Dequeue**: Remove an element from the front.
- **Peek/Front**: View the first element.
- **isEmpty()**: Check whether the queue is empty.
- **Size()**: Determine the number of elements.

Applications of Queues

- CPU Scheduling
- Printer Spooling
- Task Scheduling
- Network Packet Routing
- Ticket Reservation Systems
- Customer Service Systems
- Breadth First Search (BFS)
- Message Queues

Sample Program:

Queue Implementation Using Array

1. Initialize front = 0 and rear = -1.
2. For enqueue:
    - Check whether the queue is full.
    - Increment rear.
    - Insert the element.
3. For dequeue:
    - Check whether the queue is empty.
    - Remove the element at front.
    - Increment front.
4. Display the elements between front and rear.

Java Code:

```
import java.util.Scanner;
class ArrayQueue {

    private int[] queue;
    private int front;
    private int rear;
    private int capacity;

    ArrayQueue(int size) {
        capacity = size;
        queue = new int[capacity];
        front = 0;
        rear = -1;
    }

    void enqueue(int value) {

        if (rear == capacity - 1) {
            System.out.println("Queue Overflow");
            return;
        }

        queue[++rear] = value;
    }

    int dequeue() {

        if (front > rear) {
            System.out.println("Queue Underflow");
            return -1;
        }

        return queue[front++];
    }

    void display() {

        if (front > rear) {
            System.out.println("Queue is empty");
            return;
        }
```

```
        System.out.print("Queue: ");

        for (int i = front; i <= rear; i++)
            System.out.print(queue[i] + " ");

        System.out.println();
    }
}

public class QueueDemo {

    public static void main(String[] args) {
        ArrayQueue q = new ArrayQueue(5);
        q.enqueue(10);
        q.enqueue(20);
        q.enqueue(30);
        q.enqueue(40);
        q.display();
        System.out.println("Deleted: " + q.dequeue());
        q.display();
    }
}
```

Sample Output

Queue: 10 20 30 40
Deleted: 10
Queue: 20 30 40

Week 3 Exercises:

1. Write a Java program to implement a Queue using an array and demonstrate its application in a Printer Job Scheduling System. In the printer system, print jobs are processed in the order they are received (FIFO).
   The program should support the following operations:
   - Add a print job to the queue (Enqueue).
   - Process a print job (Dequeue).
   - Display the next print job to be processed.
   - Display all print jobs currently waiting in the queue.
   - Handle Queue Overflow and Queue Underflow conditions.

2. Write a Java program to implement a Circular Queue using an array and perform enqueue, dequeue, peek, and display operations. Demonstrate how circular queues overcome the limitations of a linear queue by reusing vacant spaces created after deletions.

3. Write a Java program to implement a Double-Ended Queue (Deque) using an array. The program should support the following operations:
    - Insert an element at the front of the deque.
    - Insert an element at the rear of the deque.
    - Delete an element from the front of the deque.
    - Delete an element from the rear of the deque.
    - Display all elements present in the deque.
    - Handle Overflow and Underflow conditions.
4. Write a Java program to implement a Priority Queue. Perform insertion, deletion, and display operations, ensuring that elements with higher priority are processed before elements with lower priority. Demonstrate the working of the priority queue using suitable test cases.

Additional Exercises:

5. Given an integer array and a window of size `k`, develop an efficient Java program using a **Deque** to determine the maximum element in every window.

    Example:

    ```
    Input:

    Array = [1, 3, -1, -3, 5, 3, 6, 7]

    k = 3
    ```

    Expected output:

    ```
    3 3 5 5 6 7
    ```

6. Design and implement a Java program for a task scheduler in which every task has:

    ```
    Task ID
    Task Name
    Priority
    Arrival Time
    Execution Time
    ```

    The scheduler should:
    - Process higher-priority tasks first.
    - Process tasks with the same priority according to arrival order.
    - Maintain waiting tasks using an appropriate queue.
    - Display the final execution sequence.

Week-5 Single Linked List

Learning Outcomes:
Upon successful completion of this laboratory session, students will be able to:

- Implement a singly linked list using Java.
- Perform insertion, deletion, and traversal operations.
- Analyze the advantages of linked lists over arrays.
- Apply linked lists to solve real-world problems.
- Develop efficient solutions using linked list operations.

A Singly Linked List (SLL) is a dynamic linear data structure consisting of nodes. Each node contains:

- Data
- Reference (link) to the next node

Unlike arrays, linked lists do not require contiguous memory locations and can grow or shrink dynamically during runtime.

Structure of a Node:

```
+------+------+
| Data | Next |
+------+------+
```

Eg: 10 → 20 → 30 → 40 → NULL

Advantages of Singly Linked List

- Dynamic size allocation.
- Efficient insertion and deletion.
- No memory wastage due to fixed size.
- Ease of implementation of stacks and queues.

Basic Operations:

1. Insertion
   - Insert at beginning
   - Insert at end
   - Insert at a specific position
2. Deletion
   - Delete from beginning
   - Delete from the end
   - Delete a given node
3. Traversal
   - Visit every node in sequence.
4. Searching
   - Find a specified element.

Sample Exercise:
Write a Java program to implement a Singly Linked List and perform the following operations:

1. Insert elements at the end.
2. Display all elements.
3. Search for an element entered by the user.

Java Code"

```
import java.util.Scanner;
class Node {
	int data;
	Node next;

	Node(int data) {
		this.data = data;
		this.next = null;
	}
}

public class SinglyLinkedList {

	Node head;
	void insert(int data)
	{
		Node newNode = new Node(data);

		if (head == null) {
		head = newNode;
		return;
		}

		Node temp = head;

		while (temp.next != null) {
		temp = temp.next;
		}

		temp.next = newNode;
	}

	void display() {
		Node temp = head;
```

```
        while (temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
        }

        System.out.println("NULL");
}

boolean search(int key) {
        Node temp = head;

        while (temp != null) {
                if (temp.data == key)
                return true;

                temp = temp.next;
        }

        return false;
        }

public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        SinglyLinkedList list = new SinglyLinkedList();
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        for(int i=0;i<n;i++) {
                list.insert(sc.nextInt());
        }

        System.out.println("Linked List:");
        list.display();
        System.out.print("Enter element to search: ");
        int key = sc.nextInt();
        if(list.search(key))
        System.out.println("Element found.");
        else
        System.out.println("Element not found.");
        }
}
```

Week-5 Exercises:

1. Write a Java program to implement a Singly Linked List and insert elements at the beginning. Display the resulting list after each insertion.
2. Write a Java program to delete a specific element from a singly linked list and display the updated list.
3. Write a Java program to count and display the total number of nodes present in a singly linked list.
4. Write a Java program to traverse a singly linked list and find the maximum element stored in the list.
5. Write a Java program to reverse a singly linked list without using an additional linked list.
6. Write a Java program to search for a given element in a singly linked list and display whether it is found or not.
7. Write a Java program to insert a new node after a specified value in a singly linked list.
8. Write a Java program to find the middle node of a singly linked list using the slow and fast pointer technique.

Additional Exercises:

9. Create a singly linked list where each node stores:
   Student ID
   Student Name
   CGPA

   Implement operations to:
   - Insert student records
   - Display all records
   - Search a student by ID
   - Delete a student record

Week-6 Double Linked List and Circular List

Learning Objectives

At the end of this laboratory session, students will be able to:

- Understand the structure of Doubly Linked Lists and Circular Linked Lists.
- Implement insertion, deletion, and traversal operations.
- Compare singly, doubly, and circular linked lists.
- Apply linked lists to solve practical problems.
- Develop menu-driven Java programs using dynamic data structures.

Doubly Linked List (DLL)

A Doubly Linked List consists of nodes containing:

- Data
- Reference to the Previous Node
- Reference to the Next Node

Structure

NULL <- [10] <-> [20] <-> [30] -> NULL

Advantages

- Traversal in both directions
- Easy insertion and deletion
- Efficient implementation of undo/redo operations

Applications

- Browser Navigation
- Music Playlist
- Undo/Redo Operations
- Image Viewer

Circular Linked List (CLL)

In a Circular Linked List, the last node points back to the first node.

Advantages

- Continuous traversal
- No NULL pointer at the end
- Efficient implementation of cyclic processes

Applications

- Round Robin Scheduling
- Multiplayer Games
- Circular Queues
- Traffic Signal Management

Sample Program

Write a Java program to implement a Doubly Linked List and perform insertion and display operations.

```
class Node {
	int data;
	Node prev, next;

	Node(int data) {
		this.data = data;
		prev = next = null;
	}
}

public class DoublyLinkedListDemo {

	Node head = null;

	void insertAtEnd(int data) {
	Node newNode = new Node(data);

	if (head == null) {
	head = newNode;
	return;
	}

	Node temp = head;

	while (temp.next != null)
	temp = temp.next;

	temp.next = newNode;
	newNode.prev = temp;
	}

	void displayForward() {
	Node temp = head;

	System.out.print("Forward Traversal: ");

	while (temp != null) {
		System.out.print(temp.data + " ");
		temp = temp.next;
	}
	}

	public static void main(String[] args) {

		DoublyLinkedListDemo dll =
		new DoublyLinkedListDemo();
```

```
        dll.insertAtEnd(10);
        dll.insertAtEnd(20);
        dll.insertAtEnd(30);
        dll.insertAtEnd(40);

        dll.displayForward();
    }
}
```

Sample Output

Forward Traversal: 10 20 30 40

Week-6 Exercises:

1. Write a Java program to implement a Doubly Linked List with the following operations:

   Insert at the beginning.

   Insert at the end.

   Delete a node.

   Display the list in the forward direction.

   Display the list in backward direction.

2. Write a Java program using a Doubly Linked List to:
   - Visit a webpage.
   - Move to the previous page.
   - Move to the next page.
   - Display the current page.

     Example:
     Google
     YouTube
     Wikipedia

3. Write a Java program using a Doubly Linked List to manage a music playlist. The program should:
   - Add songs.
   - Remove songs.
   - Move to the next song.
   - Move to the previous song.

- Display the current playlist.

4. Write a Java program to implement a Circular Linked List with the following operations:
    - Insert a node.
    - Delete a node.
    - Search for an element.
    - Display all elements.
    - Verify that the last node points back to the first node.
5. Implement a Round Robin CPU Scheduling system using a Circular Linked List.
    - The program should:
    - Add processes.
    - Execute processes cyclically.
    - Display the execution order.
    - Remove completed processes.

6. In a multiplayer game, players take turns repeatedly in a circular manner.Write a Java program using a Circular Linked List to:
    - Add players.
    - Remove a player.
    - Display the current turn.
    - Move to the next player's turn.

Week-7 Tree and its Applications

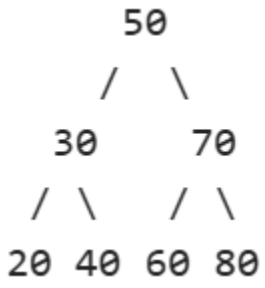
Tree

A Tree is a non-linear hierarchical data structure consisting of nodes connected by edges.

Terminology

- **Root**: Topmost node of the tree.
- **Parent**: Node having child nodes.
- **Child**: Node connected below a parent.
- **Leaf Node**: Node having no children.
- **Internal Node**: Node having at least one child.
- **Height**: Longest path from root to leaf.
- **Level**: Distance from the root.

Example:



Binary Tree

A tree in which each node has at most two children.

Operations

- Insertion
- Deletion
- Searching
- Traversal

Binary Search Tree (BST)

A Binary Tree satisfying:

- Left subtree contains values smaller than the root.
- Right subtree contains values greater than the root.

Advantages:

- Fast searching
- Ordered data storage
- Efficient insertion and deletion

Time Complexity

Operation	Average Case
Search	O(log n)
Insert	O(log n)
Delete	O(log n)

Tree Traversals

Inorder (LNR)

Left → Root → Right

Output of BST in sorted order.

Preorder (NLR)

Root → Left → Right

Used for tree copying.

Postorder (LRN)

Left → Right → Root

Used in expression tree evaluation.

Level Order

Traversal level by level using Queue.

Sample Exercise

Program: Binary Search Tree with Traversals

Algorithm

1. Create BST node structure.
2. Insert nodes recursively.
3. Perform Inorder traversal.
4. Perform Preorder traversal.
5. Perform Postorder traversal.
6. Display outputs**.**

Java Program:

```
class Node {
int data;
Node left, right;

Node(int data) {
    this.data = data;
    left = right = null;
    }
}

public class BSTDemo {

    Node root;

    Node insert(Node root, int data) {
    if(root == null)
    return new Node(data);

    if(data < root.data)
    root.left = insert(root.left, data);
    else
    root.right = insert(root.right, data);

    return root;
}

void inorder(Node root) {
    if(root != null) {
    inorder(root.left);
    System.out.print(root.data + " ");
    inorder(root.right);
```

```
        }
    }

    public static void main(String[] args) {

        BSTDemo tree = new BSTDemo();

        int arr[] = {50,30,70,20,40,60,80};

        for(int x : arr)
        tree.root = tree.insert(tree.root,x);

        System.out.println("Inorder Traversal:");
        tree.inorder(tree.root);
    }
    }
```

Sample Output

```
Inorder Traversal:
20 30 40 50 60 70 80
```

Week-7 Exercises:

1. Create a Binary Tree and write a Java program to count:
    a) Total nodes
    b) Leaf nodes
    c) Internal nodes
2. Write a recursive Java program to determine the height of a Binary Tree.
3. Create a BST and perform searching for a given key.
    a. Insert n elements.
    b. Search user-specified value.
    c. Display number of comparisons made.
4. Create a BST from user input and display:
    a. Inorder Traversal
    b. Preorder Traversal
    c. Postorder Traversal
    d. Level Order Traversal

    Verify how the outputs differ.
5. Using BST:
    a. Find smallest element.
    b. Find largest element.
    c. Display path traversed to reach both nodes.
6. Construct an expression tree for:

(A + B) * (C - D)

Perform:

- Preorder traversal
- Inorder traversal
- Postorder traversal

Explain which traversal corresponds to:

- Prefix notation
- Infix notation
- Postfix notation

Additional Exercises:

7. A department maintains student records containing:
   Roll No
   Student Name
   CGPA

Design a BST where:

- Roll Number acts as key.
- Insert new student records.
- Search a student by Roll Number.
- Display all students in sorted order of Roll Number.
- Count students with CGPA > 8.0.

Week-8: Graph Representation and Traversal

After completing this experiment, students will be able to:

1. Represent graphs using adjacency matrix and adjacency list.
2. Implement graph data structures in Java.
3. Perform Breadth First Search (BFS) traversal.
4. Perform Depth First Search (DFS) traversal.

A **Graph** is a non-linear data structure consisting of a set of vertices (nodes) and edges connecting the vertices. A graph $G = (V, E)$ has a set of vertices V and a set of edges E. An edge may be **undirected** (u, v) or **directed** $u \rightarrow v$. Other key terms: **degree** (number of edges at a vertex; in-degree and out-degree for directed graphs), **path**, **cycle**, **connected graph**, **connected component**, and **DAG** (directed acyclic graph). With $n = |V|$ and $m = |E|$, the number of edges in a simple undirected graph is at most $n(n-1)/2$.

Graph Representation

1. Adjacency Matrix

- A 2D matrix of size V × V.
- Matrix[i][j] = 1 if an edge exists between vertices i and j; otherwise 0.

2. Adjacency List

- Each vertex maintains a list of its adjacent vertices.
- Efficient for sparse graphs.

Graph Traversal

Breadth First Search (BFS)

- Visits vertices level by level.
- Uses a Queue.
- Time Complexity: **O(V + E)**

Depth First Search (DFS)

- Explores a path completely before backtracking.
- Uses a Stack or Recursion.
- Time Complexity: **O(V + E)**

Week-8 Exercises:

Write Java programs to solve the following:

1. Create an undirected graph with the following edges:
   (0,1)
   (0,2)
   (1,3)
   (2,3)
   (3,4)
   a) Create an adjacency matrix.
   b) Display the matrix.
   c) Identify the degree of each vertex.

2. Construct the graph:
   0 → 1,2
   1 → 3
   2 → 3,4
   3 → 5
   4 → 5
   5 → -
   a) Create an adjacency list.
   b) Display all adjacency lists.
   c) Count total vertices and edges.

3. Given the graph:
   A → B, C
   B → D, E
   C → F
   D → -
   E → F
   F → -
   Represent the graph using an adjacency list and perform:
   a) BFS traversal starting from A
   b) DFS traversal starting from A

4. Cities are represented as graph vertices.
   Mangalore → Udupi, Manipal
   Udupi → Kundapura
   Manipal → Karkala
   Kundapura → -
   Karkala → -
   a) Create the graph using adjacency lists.
   b) Perform BFS traversal from Mangalore.
   c) Perform DFS traversal from Mangalore.
   d) Compare the traversal orders obtained from BFS and DFS.

Week-9: Algorithm design Technique: Brute force method

Learning Outcomes

After completing this experiment, students will be able to:

1. Understand the concept of the Brute Force design technique.
2. Apply Brute Force methods to solve searching and string matching problems.
3. Analyze the time complexity of Brute Force algorithms.
4. Compare Brute Force solutions with optimized approaches.
5. Implement Brute Force algorithms in Java.

Brute Force Technique

Brute Force is a straightforward problem-solving approach that systematically checks all possible solutions until the correct one is found.

Characteristics

- Simple to understand and implement.
- Does not require sophisticated data structures.
- Guarantees finding a solution if one exists.
- Often inefficient for large inputs.

Applications

- Linear Search
- String Pattern Matching
- Traveling Salesman Problem (TSP)
- Closest Pair Problem
- Selection Sort
- Bubble Sort

Sample Exercise

Given an integer array: 45, 12, 67, 23, 89, 34, 56

Use the Brute Force technique (Linear Search) to find whether the key value **89** exists in the array.

Program:

```
public class LinearSearch {
    public static void main(String[] args) {
        int arr[] = {45, 12, 67, 23, 89, 34, 56};
        int key = 89;
```

```
        int position = -1;
        for(int i = 0; i < arr.length; i++) {
            if(arr[i] == key) {
                position = i;
                break;
            }
        }
        if(position != -1)
            System.out.println("Element found at index " + position);
        else
            System.out.println("Element not found");
    }
}
```

Week-9 Exercises:

1. Implement Linear Search. Search for the element 60. Display the position of the element. Determine the number of comparisons performed.
2. Implement the Brute Force String Matching algorithm. Search for the pattern in the text. Display the starting position of the pattern. Count the number of character comparisons.
3. Implement Selection Sort. Display the array after each pass. Determine the total number of comparisons. Display the sorted array.
4. Implement Bubble Sort. Display the array after each pass. Determine the total number of comparisons. Display the sorted array.
5. Consider the points:

(2,3)
(12,30)
(40,50)
(5,1)
(12,10)
(3,4)

Compute the distance between every pair of points. Identify the closest pair. Display the minimum distance.

Week 10: Algorithm design Technique: Divide and conquer method

Learning Outcomes

After completing this lab, students will be able to:

- Understand the Divide and Conquer paradigm.
- Develop recursive solutions using problem decomposition.
- Analyze time complexity of Divide and Conquer algorithms.
- Implement searching, sorting, and mathematical algorithms using Java.
- Compare Divide and Conquer solutions with iterative approaches.

Introduction

Divide and Conquer is an algorithm design strategy that solves a problem by:

1. **Divide**: Break the problem into smaller subproblems.
2. **Conquer**: Solve the subproblems recursively.
3. **Combine**: Merge the solutions of subproblems to obtain the final result.

Examples

- Binary Search
- Merge Sort
- Quick Sort
- Closest Pair of Points
- Strassen Matrix Multiplication

Sample Exercise: Binary Search using Divide and Conquer

Given a sorted array of integers, search for a given element using Binary Search.

Algorithm

1. Find the middle element.
2. Compare the key with the middle element.
3. If equal, return the position.
4. If smaller, search the left half.
5. Otherwise, search the right half.
6. Repeat recursively until the element is found or the range becomes empty.

Java Program:

```
public class BinarySearch {
   static int binarySearch(int arr[], int low, int high, int key) {
      if (low <= high) {
         int mid = (low + high) / 2;
         if (arr[mid] == key)
            return mid;
         if (key < arr[mid])
            return binarySearch(arr, low, mid - 1, key);
         return binarySearch(arr, mid + 1, high, key);
      }
      return -1;
   }
   public static void main(String[] args) {
      int arr[] = {10, 20, 30, 40, 50, 60, 70};
      int key = 50;
      int result = binarySearch(arr, 0, arr.length - 1, key);
      if (result != -1)
         System.out.println("Element found at index " + result);
      else
         System.out.println("Element not found");
   }
}
```

Time Complexity: O(log n)

Week-10 Exercises:

1. Implement Merge Sort using Divide and Conquer to sort an array of integers in ascending order.
2. Implement Quick Sort using Divide and Conquer.
3. Find the maximum and minimum elements in an array using Divide and Conquer.
4. Write a Java program to determine the height of a binary tree using the Divide and Conquer technique.

Additional Exercises:

5. Given an array of integers, find the pair having the minimum absolute difference using a Divide and Conquer approach.

Algorithm design Technique: Dynamic Programming

After completing this laboratory, students will be able to:

- Understand the principles of Dynamic Programming.
- Differentiate between recursion, Divide and Conquer, and Dynamic Programming.
- Apply memoization and tabulation techniques.
- Develop efficient solutions for optimization problems.
- Analyze the time and space complexity of Dynamic Programming algorithms.

Dynamic Programming (DP) is an algorithm design technique used to solve problems exhibiting:

1. **Optimal Substructure** - The optimal solution can be constructed from optimal solutions of subproblems.
2. **Overlapping Subproblems** - The same subproblems are solved multiple times.

Approaches

Memoization (Top-Down)

- Uses recursion.
- Stores previously computed results.

Tabulation (Bottom-Up)

- Uses iterative computation.
- Builds solutions from smaller subproblems.

Applications

- Fibonacci Numbers
- Knapsack Problem
- Longest Common Subsequence (LCS)
- Matrix Chain Multiplication
- Shortest Path Problems

Sample Exercise:

Write a Java program to find the nth Fibonacci number using Dynamic Programming.

Algorithm

1. Create an array dp[].
2. Initialize dp[0] = 0 and dp[1] = 1.
3. Compute all Fibonacci numbers iteratively.
4. Return dp[n].

Java Program

```
public class FibonacciDP {
   public static int fibonacci(int n) {
      if (n <= 1)
         return n;
      int[] dp = new int[n + 1];
      dp[0] = 0;
      dp[1] = 1;
```

```
        for (int i = 2; i <= n; i++)
            dp[i] = dp[i - 1] + dp[i - 2];
        return dp[n];
    }
    public static void main(String[] args) {
        int n = 10;
        System.out.println("Fibonacci Number = " + fibonacci(n));
    }
}
```

Week-11 Exercises:

1. Given a set of items, each having a weight and profit, determine the maximum profit that can be obtained without exceeding the knapsack capacity.
2. Given two strings, find the length of the Longest Common Subsequence.

Week-12:Algorithm design Technique: Greedy Technique

Learning Outcomes

After completing this laboratory, students will be able to:

- Understand the Greedy algorithm design paradigm.
- Identify problems suitable for Greedy solutions.
- Apply local optimal choices to obtain global solutions.
- Implement Greedy algorithms using Java.
- Analyze the efficiency of Greedy-based solutions.

A Greedy Algorithm builds a solution step by step by selecting the best available choice at each stage without reconsidering previous decisions.

Characteristics of Greedy Algorithms

1. **Greedy Choice Property**
   - A locally optimal choice leads to a globally optimal solution.
2. **Optimal Substructure**
   - An optimal solution can be constructed from optimal solutions of subproblems.

Applications

- Activity Selection Problem
- Fractional Knapsack
- Huffman Coding
- Job Sequencing with Deadlines
- Minimum Spanning Tree (Prim's and Kruskal's Algorithms)
- Dijkstra's Shortest Path Algorithm

Sample Exercise:

Given a connected weighted graph, find the Minimum Spanning Tree (MST) using Prim's Algorithm. The MST should connect all vertices with the minimum possible total edge weight.

Algorithm

1. Start with any vertex as the source.
2. Mark the source vertex as visited.
3. Select the minimum-weight edge connecting a visited vertex to an unvisited vertex.
4. Add the selected edge to the MST.
5. Mark the new vertex as visited.
6. Repeat steps 3–5 until all vertices are included in the MST.

Java Program:

```
public class PrimsAlgorithm {
   static final int V = 4;
   int minKey(int key[], boolean mstSet[]) {
      int min = Integer.MAX_VALUE, minIndex = -1;
```

```
    for (int v = 0; v < V; v++) {
        if (!mstSet[v] && key[v] < min) {
            min = key[v];
            minIndex = v;
        }
    }
    return minIndex;
}
void primMST(int graph[][]) {
    int parent[] = new int[V];
    int key[] = new int[V];
    boolean mstSet[] = new boolean[V];
    for (int i = 0; i < V; i++) {
        key[i] = Integer.MAX_VALUE;
        mstSet[i] = false;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = true;
        for (int v = 0; v < V; v++) {
            if (graph[u][v] != 0 &&
                !mstSet[v] &&
                graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
    printMST(parent, graph);
}
void printMST(int parent[], int graph[][]) {
    int totalCost = 0;
    System.out.println("Edge \tWeight");
    for (int i = 1; i < V; i++) {
        System.out.println(
            parent[i] + " - " + i +
            "\t" + graph[i][parent[i]]
        );
        totalCost += graph[i][parent[i]];
    }
    System.out.println("Total Cost = " + totalCost);
```

```
    }
    public static void main(String[] args) {
        int graph[][] = {
                {0, 2, 6, 3},
                {2, 0, 0, 8},
                {6, 0, 0, 5},
                {3, 8, 5, 0}
        };
        PrimsAlgorithm p = new PrimsAlgorithm();
        p.primMST(graph);
    }
}
```

Week-12 Exercises:

1. Given a weighted graph representing cities and connection costs, find the minimum cost required to connect all cities.
2. Given weights and profits of items, maximize the profit without exceeding the knapsack capacity. Items can be divided.